CHOREO

GCU 算子编程语言及编译器
为提高算子开发效率而生

简化数据搬运编程 | 提高程序安全性 | 支持动态形状

Choreo Pre-Beta Release 发布
Choreo 内部试用开启

为什么要设计 Choreo？

数据搬运（数据流）编程是算子编程日常工作中的一个核心。在现有的 GCU 算子编程工作流中，其使用底层的 C++ 编程模型 Topscc，配合 GCU 软件栈特有的 DTE（Data Transfer Engine）API 实现数据搬运的编程。程序员往往既需要打磨算法，又要考虑高效地在硬件上移动数据，还要关注代码中实现的细节。因此，任何能够降低算子开发中的高复杂性的工具都可以提高算子开发效率。

这促使我们考虑开发 Choreo 来简化编程细节的部分。在初期，Choreo 的特性设计都着重于简化数据搬运编程，试图降低 GCU 上算子编程的门槛。

随着研发的深入，我们也关注到了算子开发中的其它痛点：开发者很多开发时间都被花在了代码调试上。算子的代码有时也会面临可维护性的问题 — 这些工程问题常常会拖累算子开发的进展。

而带来这些问题的原因，比如不安全的下标访问，过于灵活的编程方法等等都属于现有工具较难捕捉的问题。在现实的工程实践中，使用多个层面的检测工具，并辅以算子编程专项指导，可以有限地避免问题产生。但编程工具在这个层面能够做到的安全性却是其它方法难于取代的。这也促使 Choreo 的设计更加关注代码安全，以及为促进代码可维护性而实现的编程限制等方面。

CHOREO 是为机器学习的算子编程，尤其是 GCU 上需要进行跨存储层级的数据搬运而全新打造的一门领域特定编程语言（Domain Specific Language, DSL）。程序员可以使用 Choreo 编译器进行 GCU 上的 Choreo-C++ 异构程序的构建，以实现高性能算子程序。

Choreo 着眼于提高算子编程全过程中的开发效率。使用 Choreo 进行算子编程，可以获得如下好处：

- 更为直观的编程方法，更容易维护的代码，以及更少的代码量：
  - Choreo 构建了数据切分（Tiling）以及数据搬运操作的高级编程抽象。这简化了底层的基于数据下标的编程，提高了代码的可维护性；
  - Choreo 使用高级抽象简化异构程序的编程，使得程序员无需关心异构程序中较为事务性的工作，如设备存储管理，设备代码启动等；
  - Choreo 实施了栈式的 ScratchPad 存储（L2 及 L1）管理，并实现了自动存储分配优化。
- 更为安全的代码：Choreo 构建了数据形状的高级编程抽象，并通过静态形状推导以及轻量级的运行时检查，使得不安全的数据切分（如除零），数据越界访问，不合法的 DTE 硬件编程，不安全的异步代码等问题得到系统性的安全检查；
- 更为强大动态形状支持：Choreo 针对机器学习场景中常见的动态形状数据，实现了用户友好的符号化形状支持。而且动态形状的代码也能够得到系统性的安全检查。

此外，Choreo 的一些异步编程的语言特性也在持续迭代开发中，这些特性会使得 Choreo 能够支持更多应用场景。

值得一提的是，在 Choreo 的设计中，所有编程抽象都是零开销的。这意味着 Choreo 编译出的代码可以与底层编程模型 Topscc 编写的程序达到同等的执行效率。

CHOREO

多开发场景的支持

通过提供完善的编译器功能和多样化的输出形式，Choreo 编译器能够支持多种开发使用场景，包括:

- 异构的可执行文件。该模式适用于端到端的算子开发工作流程。
- 异构的可链接模块。该模式适用于将 Choreo 编译的模块集成到已有系统中。
- 等效的 *Topscc* 的源代码。该模式下，程序员可以将 Choreo 作为一个快速的 Topscc 代码生成器而使用。
- 可执行的 *bash* 脚本。在这种模式下，bash 脚本中包含了等效的 Topscc 源码以及编译、执行的相关脚本。程序员可以调整脚本中的生成内容进行 Topscc 级别的代码调试。

程序员可以根据实际的开发环境选择让 Choreo 产生不同的输出以适应特定的工作场景。

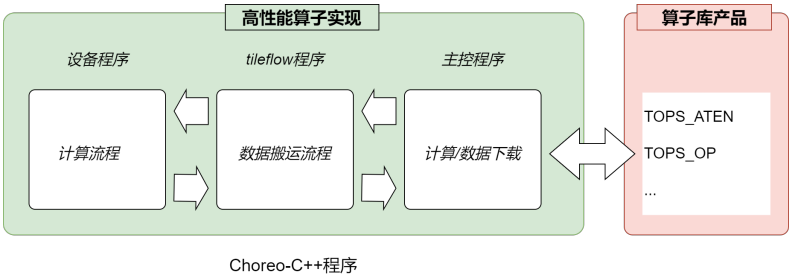
猜猜看：Choreo 代码的语义

相比于 Topscc 代码，Choreo 中包含了一段独有的 Tileflow 程序，这通常也是 Choreo 程序的核心代码。下面的程序片段就是一个例子：

```
__co__ auto ele_add(s32 [6, 17, 64] lhs, s32 [6, 17, 64] rhs) {
  s32 [lhs.span] output;

  parallel p by 6 {
    foreach index in [17, 4] {
      lf = dma.copy.async lhs.chunkat(p, index) => local;
      rf = dma.copy.async rhs.chunkat(p, index) => local;
      wait lf, rf;

      local s32 [lf.span] l1_out;
      call kernel(lf.data, rf.data, l1_out, |lf.span|);

      dma.copy l1_out => output.chunkat(p, index);
    }
  }
  return output;
}
```

通过与大模型交流测试发现，大模型可以在没有了解 Choreo 语法的情况下大致了解程序功能，甚至生成 Choreo 代码。读者你猜出了多少呢？

更好的算子开发体验



了解了 Choreo 编写算子代码的各种好处，我们可以将 Choreo 集成进 GCU Tops 软件栈的产品中来改善算子编程体验。（原型系统已经验证，整合工作正在开展，欢迎大家参与其中）

深入了解 Choreo

如果想深入了解 Choreo，我们准备了相关的文档、视频教程供您学习：

- 网页教程：http://choreo.enflame.cn/
- 视频教程：https://learning.enflame-tech.com/course/144

参与到 Choreo 的开发中

无论是汇报问题，或是建议新的 Choreo 特性，都可以通过在 Choreo 项目主页创建 issue 的方式达成：http://git.enflame.cn/xiaofeng.guan/choreo/-/issues

Choreo 的**项目主页**在：http://git.enflame.cn/xiaofeng.guan/choreo, 其包含了编译使用 Choreo 的说明

Choreo Pre-Beta 内部试用召集

Choreo Pre-Beta 下载链接：http://git.enflame.cn/xiaofeng.guan/choreo/-/packages/65

欢迎汇报问题，分享试用体验，以及提出建议。如有需要请联系：

- 官孝峰：xiaofeng.guan@enflame-tech.com
- 石恒：heng.shi@enflame-tech.com